

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

*I **HEMASRINITHA (192572179)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Hemasrinitha P

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

QUESTION 1:-

REVERSE ENGINEERING A RECOMMENDATION SYSTEM

A large e-commerce platform recommends products by analyzing customer activities such as searches, clicks, purchases, ratings, and browsing history. The system continuously processes this real-time data using **PySpark and RDD operations** to identify user interests and generate suitable recommendations.

1. Data Collection

The system collects data from different sources such as product searches, clicks, purchases, cart additions, ratings, and browsing history. Real-time user activity is captured as events and sent to the processing system. Product information such as category, price, brand, and popularity is also collected.

2. Data Processing using PySpark

PySpark is used to process the large amount of collected data efficiently. The data can be converted into RDDs and cleaned by removing duplicate or invalid records. Operations such as `map()`, `filter()`, `groupByKey()`, and `reduceByKey()` are used to transform the raw data into useful user-product relationships.

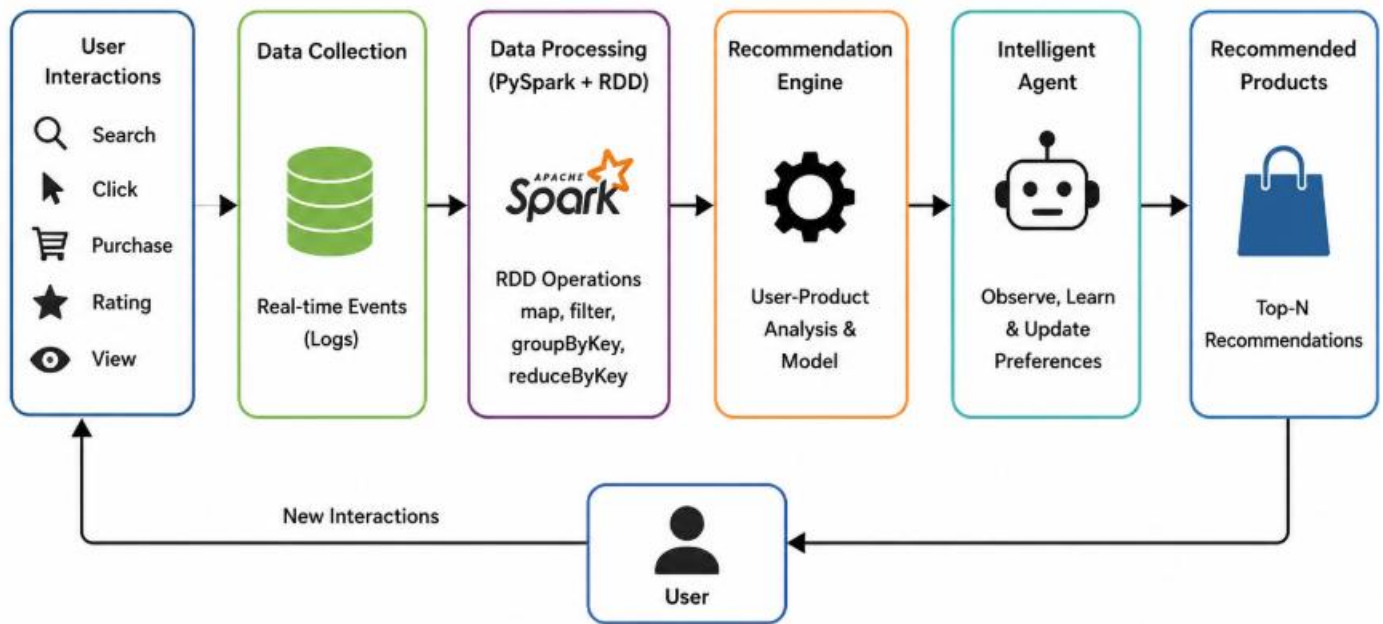
3. Recommendation Generation

After processing, the system identifies products that are related to a user's interests. For example, if a customer frequently searches for laptops, the system can identify similar laptops and related accessories. RDD operations help calculate product popularity, user preferences, and frequently purchased product combinations.

4. Intelligent Agent

An intelligent recommendation agent observes user activities and makes decisions about which products should be displayed. It continuously monitors new interactions and updates the user's preference profile. When the user's interests change, the agent can dynamically modify the recommended products instead of relying only on old data.

5. System Architecture



6. Dynamic Updating and Conclusion

The recommendation system works as a continuous feedback loop. Every new search, click, or purchase provides additional information to the intelligent agent, which updates the user's preference information and improves future recommendations. Thus, **PySpark provides scalable data processing, RDDs handle transformations, and the intelligent agent makes dynamic recommendation decisions** based on changing user behaviour.

QUESTION 2:-

REVERSE ENGINEERING A SMART TRAFFIC MANAGEMENT SYSTEM

1. Real-World Traffic Problem

A city-wide traffic management system must handle changing vehicle density, congestion, accidents, and peak-hour traffic. Fixed-time signals may cause unnecessary waiting when one road is crowded while another is almost empty. The AI system therefore collects real-time traffic information and dynamically changes signal timings to reduce congestion, waiting time, and fuel consumption. Since traffic conditions change every few seconds, decisions must be made quickly without affecting road safety.

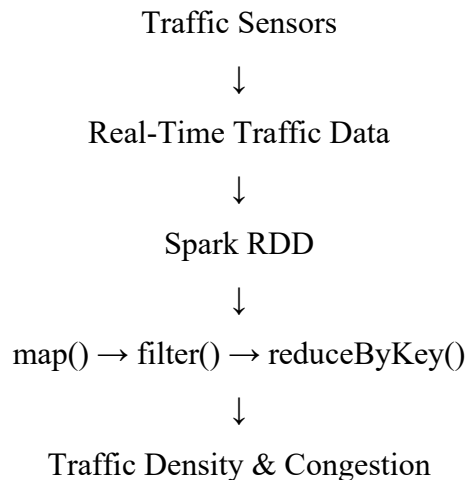
2. Real-Time Data Collection and RDD Processing

Traffic cameras, vehicle sensors, GPS data, and IoT devices provide information such as vehicle count, lane occupancy, and average speed. This data is converted into streams and processed using Apache Spark and RDD-based operations.

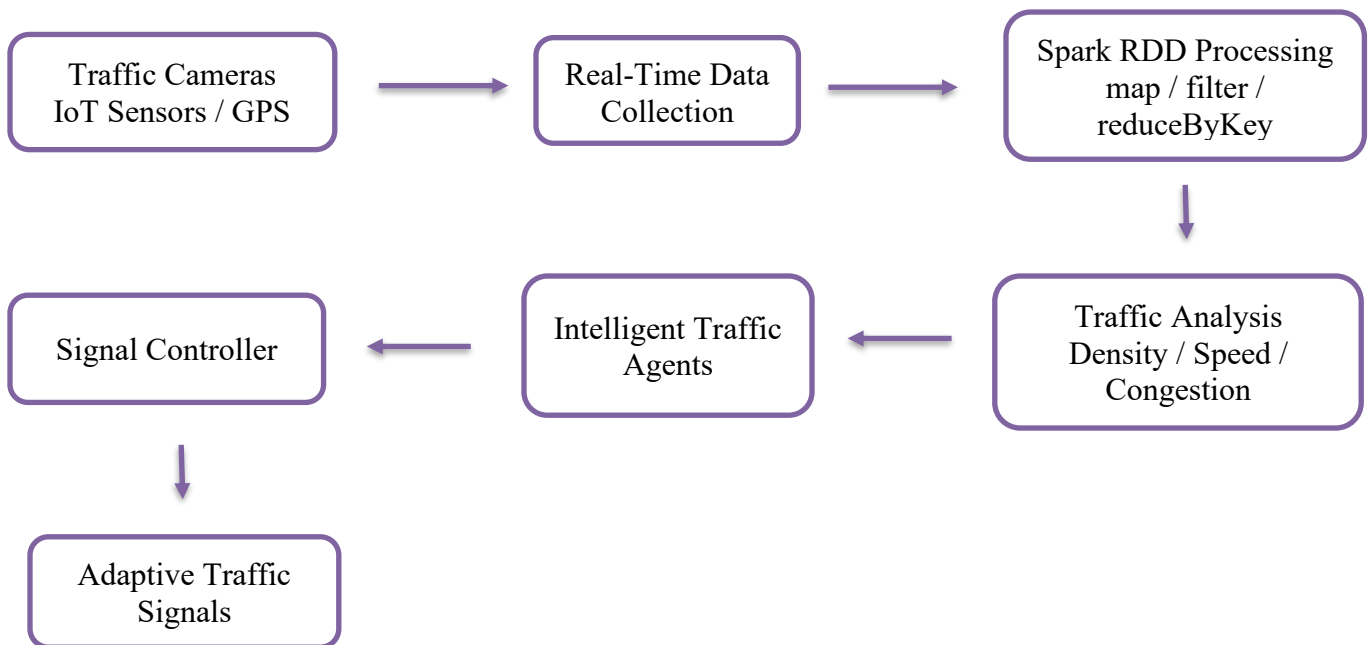
Typical RDD operations include:

- `map()` – converts raw sensor data into vehicle-density records.
- `filter()` – removes invalid or incomplete readings.
- `reduceByKey()` – combines vehicle counts for each road or junction.
- `groupByKey()` – groups traffic information by junction or lane.

Example:



3. Smart Traffic System Architecture



4. Interaction Between Intelligent Agents

Each major junction can be treated as an intelligent traffic agent. The agent observes vehicle density, queue length, signal status, and congestion level, then selects the most suitable signal action. Agents can communicate with neighbouring junctions so that clearing traffic at one signal does not create congestion at

the next signal. For example, if Junction A detects heavy traffic moving toward Junction B, it can communicate this information to B so that B prepares an appropriate signal phase.

5. Decision-Making Under Time Constraints

The agent compares the current traffic conditions and selects an action such as extend green time, reduce green time, switch the signal, or maintain the current phase. A simple congestion score can be calculated as:

$\text{Congestion Score} = \text{Vehicle Count} \times \text{Average Waiting Time}$

For example, if 40 vehicles have an average waiting time of 30 seconds:

$\text{Congestion Score} = 40 \times 30 = 1200$

A higher score indicates that the junction requires faster intervention. The decision must be completed within seconds, while safety constraints such as minimum green time, yellow time, pedestrian crossing time, and emergency-vehicle priority are maintained.

6. Performance Evaluation and Practical Implementation

The reverse-engineered system can be implemented using Python, PySpark, RDDs, traffic sensors, cameras, and a signal-control interface. Its performance can be evaluated using average waiting time, queue length, vehicle throughput, and response time. The AI approach should be compared with fixed-time signals to verify whether congestion and waiting time are reduced. Continuous monitoring also allows the system to detect abnormal traffic conditions and dynamically adjust signal timings, making the solution practical for smart-city traffic management.

QUESTION 3:-

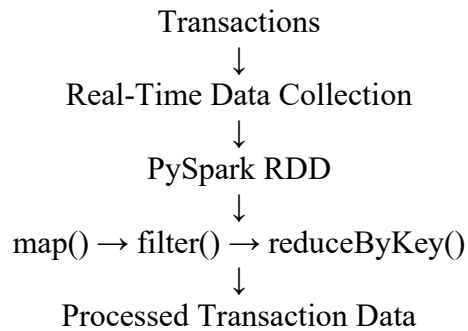
REVERSE ENGINEERING A FRAUD DETECTION PIPELINE

1. Banking Fraud Detection Problem

A banking system must identify suspicious transactions within milliseconds while allowing genuine transactions to proceed normally. The system monitors transaction amount, location, time, frequency, device information, and previous transaction patterns. Since thousands of transactions may arrive simultaneously, the system requires distributed processing and fast decision-making.

2. Real-Time Data Collection and Processing

Transaction data is collected from ATMs, mobile banking, cards, and online payments. The incoming data is distributed across multiple processing nodes using **PySpark**. RDDs are used to process large volumes of transactions in parallel.

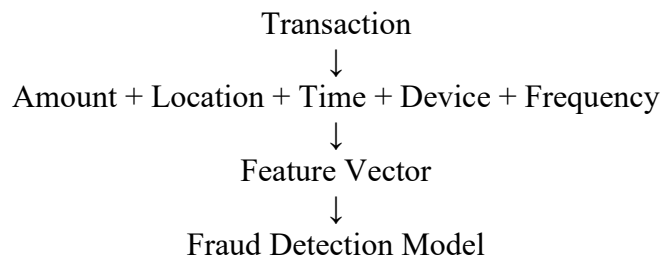


- map() transforms raw transaction records.
- filter() removes invalid or incomplete transactions.
- reduceByKey() can aggregate transaction activity by customer or account.
- RDD processing allows multiple transactions to be analyzed simultaneously.

3. Feature Extraction

The system converts raw transaction information into meaningful features for fraud detection. Important features may include transaction amount, transaction frequency, location difference, transaction time, device information, and deviation from the customer's normal spending pattern.

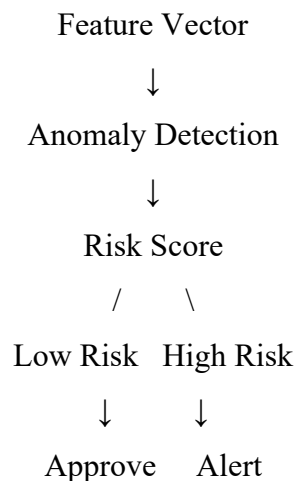
For example:



A transaction with an unusually high amount combined with a new location and unfamiliar device may receive a higher risk score.

4. Anomaly Detection

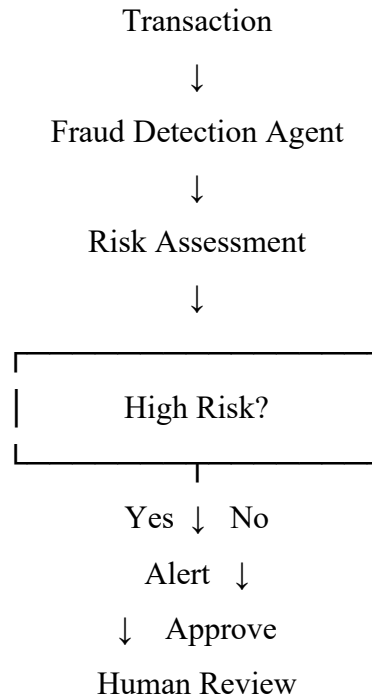
The AI model compares each transaction with normal transaction patterns. A risk score can be generated based on how different the transaction is from expected behaviour.



For example, a transaction with a risk score above a predefined threshold can be classified as suspicious. The system can use statistical methods, machine-learning models, or behavioural analysis to identify unusual patterns.

5. Agent-Based Alert Mechanism

Intelligent agents continuously monitor the processed transactions and respond to suspicious activities. A Fraud Detection Agent evaluates the risk score, while an Alert Agent generates notifications when the risk exceeds the allowed threshold. A Response Agent can temporarily block or hold a suspicious transaction for further verification.



6. Distributed Pipeline and Performance

The complete system works as a continuous distributed pipeline where transactions are collected, transformed, analysed, and classified within milliseconds. PySpark RDDs provide parallel processing, while feature extraction converts raw data into useful inputs and anomaly detection identifies suspicious behaviour. Agent-based alerts then provide immediate responses. Performance can be evaluated using detection accuracy, false-positive rate, processing latency, and number of fraudulent transactions detected, ensuring that the system remains both fast and reliable.

QUESTION 4:-

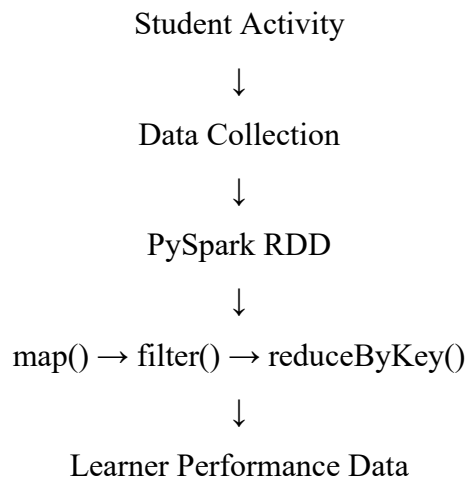
REVERSE ENGINEERING A PERSONALIZED LEARNING SYSTEM

1. Understanding the Learning Environment

An online learning platform continuously observes how students interact with learning materials. It collects information such as quiz scores, assignment marks, time spent on topics, completed lessons, mistakes, and participation. This information helps the system understand whether a learner is struggling, progressing normally, or ready for more advanced content.

2. Learner Data Processing Using RDDs

The collected learner data can be processed using **PySpark RDDs** to handle information from a large number of students in parallel. Raw records are cleaned and transformed into useful learning information.

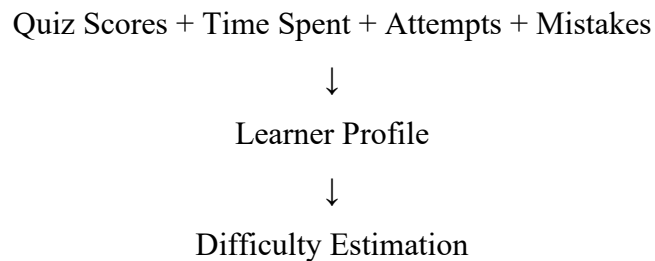


- map() converts activity records into student-performance pairs.
- filter() removes incomplete or invalid records.
- reduceByKey() calculates values such as average scores or total activity for each student.
-

3. Learner Performance and Feature Extraction

The processed data is converted into meaningful features such as **accuracy, learning speed, engagement level, topic-wise performance, and number of repeated mistakes**. These features help determine the learner's current knowledge level.

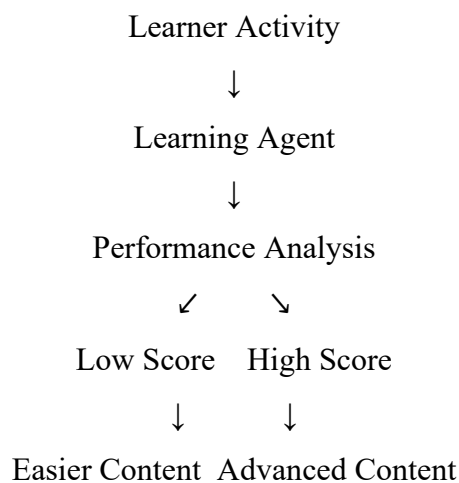
For example:



A student with high accuracy and low completion time may be considered ready for advanced content, while repeated mistakes may indicate the need for easier lessons or additional practice.

4. Intelligent Agents for Adaptive Learning

An intelligent Learning Agent observes the learner's performance and selects suitable learning content. It can increase difficulty when the student performs well and reduce difficulty when the student struggles. The agent continuously updates the learner profile based on new activities rather than using a fixed learning path.

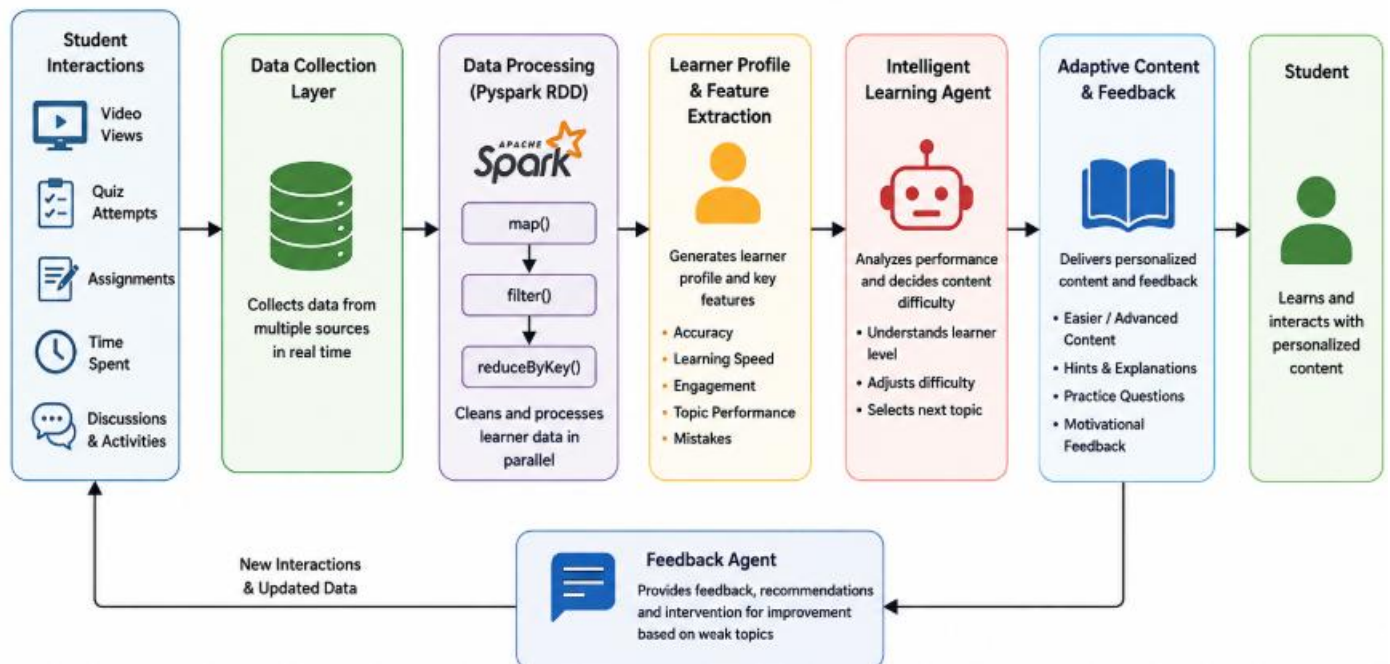


5. Feedback and Recommendation Mechanism

A **Feedback Agent** provides immediate responses after quizzes, assignments, and activities. It can identify weak topics and recommend revision materials, additional questions, or suitable videos. Positive feedback can also be provided when the learner achieves improvement, helping maintain engagement.

6. Adaptive Learning Architecture and Evaluation

The system operates as a continuous feedback cycle. **PySpark RDDs** provide scalable learner-data processing, while intelligent agents analyse performance and dynamically select content difficulty and feedback. The system can be evaluated using quiz improvement, completion rate, engagement, learning time, and recommendation accuracy to determine whether personalized learning is actually improving student outcomes.



QUESTION 5:-

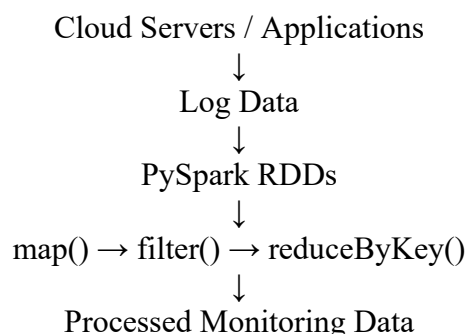
REVERSE ENGINEERING A CLOUD-BASED AI MONITORING SYSTEM

1. Understanding the Cloud Monitoring Problem

A cloud platform must continuously monitor servers, applications, networks, and storage resources. Large volumes of logs are generated from CPU usage, memory consumption, response time, errors, and network traffic. The system must detect abnormal behaviour quickly and automatically increase or decrease resources according to workload. This reduces service downtime, improves performance, and avoids unnecessary cloud costs.

2. Large-Scale Log Processing with PySpark

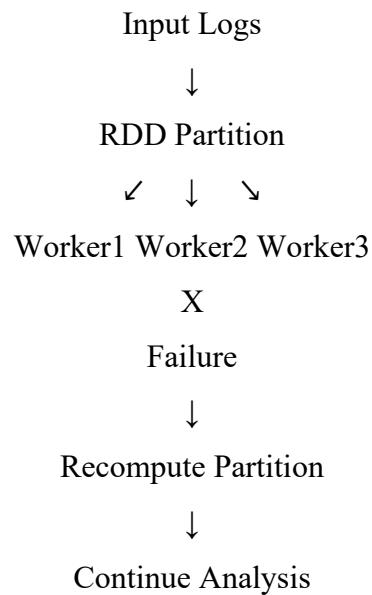
Logs from different cloud services are collected and sent to a distributed processing system. **PySpark** processes these logs in parallel, allowing thousands or millions of records to be analysed efficiently.



- `map()` extracts useful information such as CPU usage and error count.
- `filter()` removes invalid or unnecessary log records.
- `reduceByKey()` aggregates data by server, application, or time period.
-

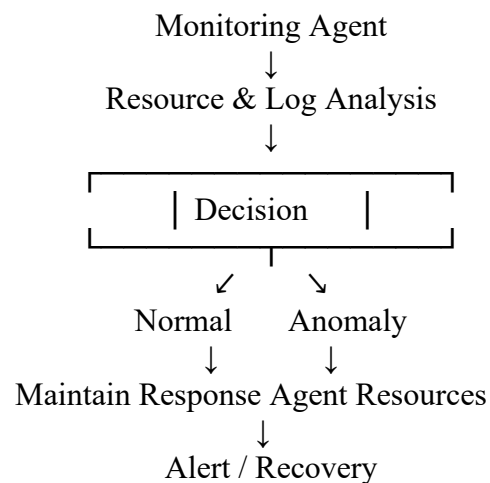
3. RDD-Based Fault Tolerance

RDDs support fault tolerance because Spark can reconstruct lost data from its transformation history (lineage) instead of depending on a single processing node. If one worker fails while processing cloud logs, the required RDD partitions can be recomputed on another available worker. This makes the monitoring pipeline more reliable and suitable for distributed cloud environments.



4. Intelligent Monitoring and Scaling Agents

The system can use multiple intelligent agents with different responsibilities. A Monitoring Agent observes system metrics and logs, while a Scaling Agent decides when additional CPU, memory, or virtual machines are required. An Anomaly Detection Agent identifies unusual behaviour such as sudden traffic increases, repeated errors, or abnormal resource consumption.



5. Automatic Scaling and Anomaly Response

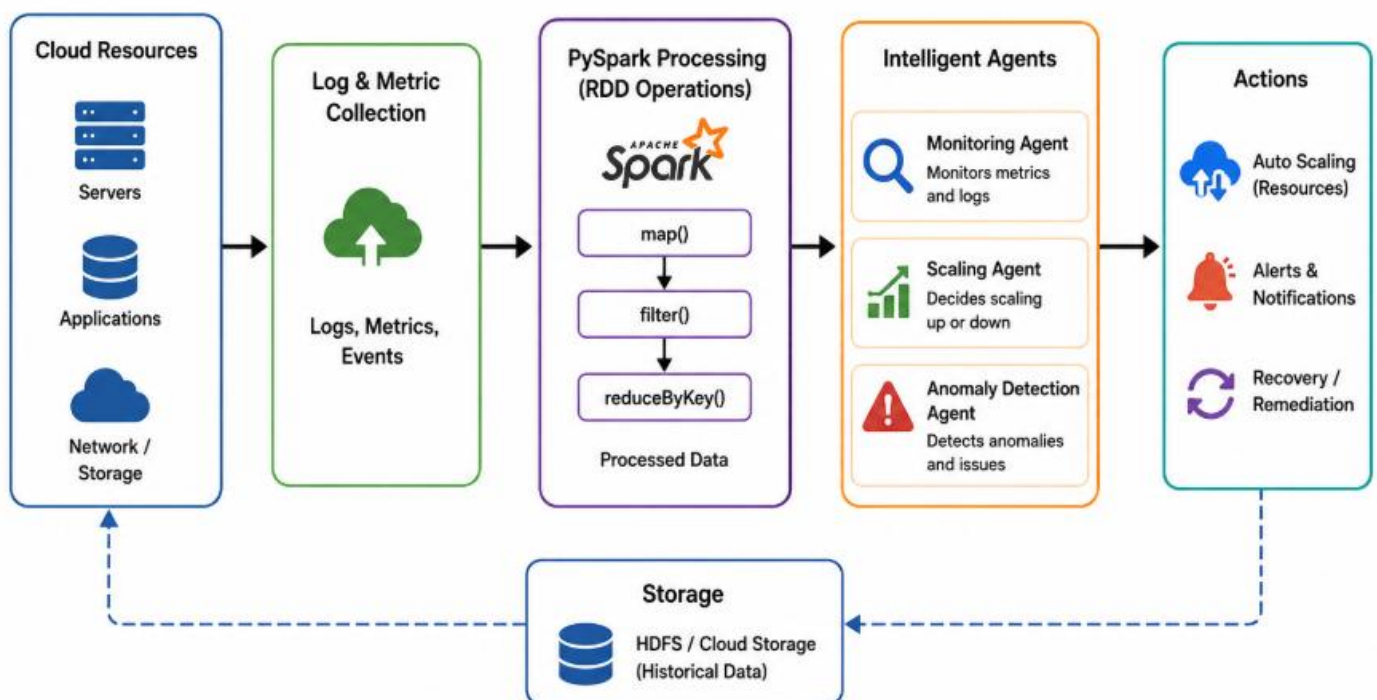
When the agents detect that resource usage has crossed a defined limit, the Scaling Agent can request additional cloud resources. When the workload decreases, unnecessary resources can be released. If an anomaly is detected, the Response Agent can generate an alert, isolate the affected service, restart a failed component, or notify an administrator.

For example:

CPU > 80% → Scaling Agent → Increase resources

Error rate suddenly increases → Anomaly Agent → Generate alert

6. Complete Architecture and Performance Evaluation



The system can be evaluated using anomaly detection accuracy, processing time, resource utilization, scaling response time, and system availability. PySpark provides scalable log processing, RDD lineage provides fault recovery, and intelligent agents coordinate monitoring, automatic scaling, and anomaly response. Together, these components create a reliable cloud monitoring system capable of responding to changing workloads in real time.